OBOOK5

Offline Dictionary Mod

Public Step-by-Step Reproduction Guide

Enable the built-in dictionary UI, pass the selected word automatically, and install custom offline dictionaries

Scope
This guide documents the successful path only. It deliberately omits the failed experiments and debugging history. It is written for owners of the exact tested OBOOK5 build who want a reproducible, public-friendly procedure.

Important legal / safety note
Use this only on a device you own or are authorized to modify. Do not publish or redistribute OBOOK proprietary firmware or binaries. The recommended GitHub repository contains patch/build scripts only; every user supplies their own exact official firmware locally.

Validated target: OBOOK5 / epd86s2e / official base V1.3

Contents

1. What this mod actually does
2. Exact tested hardware and firmware - verify before doing anything
3. What you need on the computer
4. The complete process at a glance
5. Obtain and verify the official V1.3 OTA image
6. Build the one-time owner bridge OTA
7. Redirect the OBOOK updater to your local OTA server
8. Install and verify the owner startup bridge
9. Export the stock reader libraries from the device
10. Build the Search -> Dictionary patch
11. Install Search -> Dictionary
12. Build the selected-word autofill patch
13. Install autofill v2
14. Verify the complete dictionary flow
15. Understand the dictionary database format
16. Create a new dictionary from CSV / TSV / JSON / JSONL
17. Install a custom dictionary
18. Switch between installed dictionaries
19. Merge multiple dictionaries
20. What to do with a pre-existing .dict file
21. Morphology, plurals, conjugations, case and aliases
22. Recovery and rollback
23. Recommended GitHub repository structure
24. What you should and should not publish
25. Quick reference checklist
Appendix A. Tested hashes and paths
Appendix B. What the binary patches change
Appendix C. Owner bridge logic
Appendix D. Dictionary schema and sample data

1. What this mod actually does

The OBOOK5 already contains a dictionary engine and dictionary window in the stock firmware. This project does not install a brand-new dictionary application from scratch. Instead, it enables and fixes the path between text selection in the reader and the dictionary UI that is already present.

Before

```
Long-press a word
    |
    v
Note | Mark | Search
                 |
                 v
        in-book text search
```

After

```
Long-press a word
    |
    v
Note | Mark | Dictionary
                 |
                 v
selected QString
                 |
                 v
DictionaryView::setWord(selected)
                 |
                 v
DictionaryView::search()
                 |
                 v
stock dictionary window + result
```

The final result was physically validated on the target OBOOK5: the selected word is automatically inserted and searched without manual typing.

Key idea
The dictionary engine was already there. The mod connects the existing reader selection flow to the existing dictionary UI, then lets you install your own SQLite dictionary databases.

2. Exact tested hardware and firmware

These byte patches are intentionally strict. If the hashes do not match, stop. Do not try to force the offsets onto a different build.

Item	Tested value
Product	OBOOK5
Board/model string	epd86s2e
SoC	Ingenic X2000 / XBurst II
CPU architecture	32-bit little-endian MIPS32 Release 2
Kernel	Linux 4.4.94
UI framework	Qt for Embedded Linux 4.7.4
Official base firmware	V1.3
System pack observed	2026-05-18 01:13:48
Display	480 x 800 e-ink

Required hashes

```
Official V1.3 OTA image:
8ced2385925ac03b0f6a82aa73a8e7bc

Stock libeinkfbreader.so.1.0.0:
97f249f7a36b773582e78e80896e993b

Final Search -> Dictionary libeinkfbreader:
86cd2535d141d13cb7abe4fd60746326

Stock libdictionaryview.so.1.0.0:
e18ae4624cb2bdf1edc041f203757b5d

Final autofill-v2 libdictionaryview:
02ef37b7d69995475b8cf5406ed4b053
```

Hard stop condition

If your stock library hash differs from the value above, do not apply the byte patch. A different firmware build may have different code and offsets.

3. What you need on the computer

Recommended setup

- Windows PC with Wi-Fi and USB.
- Windows Mobile Hotspot, or another local network where you control DNS.
- WSL Ubuntu for the patch build steps.
- Python 3.
- Clang and LLVM tools.
- The public script repository / companion kit.
- Your own official OBOOK5 V1.3 OTA image.

Install build tools in WSL

```
sudo apt update
sudo apt install python3 clang llvm tar gzip coreutils
```

Why WSL?

The Search -> Dictionary patch contains a small MIPS32r2 assembly hook. Clang can assemble it directly when invoked with the MIPS target. The autofill patch itself is pure Python and does not need a compiler.

Windows-side requirements

- Python 3 is useful for the local OTA server.
- Allow inbound TCP port 8085 through Windows Firewall while doing the controlled OTA.
- The OBOOK and the PC must be on the same controlled network.

4. The complete process at a glance

```
PHASE A - one-time bootstrap
official V1.3 OTA
    -> build owner-bridge OTA
    -> local OTA server
    -> OBOOK installs owner startup bridge

PHASE B - reader integration
extract stock libeinkfbreader
    -> build validated Search -> Dictionary library
    -> install via MTP + one-shot bridge

PHASE C - selected-word autofill
extract stock libdictionaryview
    -> build validated autofill-v2 library
    -> install via MTP + one-shot bridge

PHASE D - dictionary data
create / convert .dict SQLite database
    -> copy to OBOOK
    -> install/select via one-shot bridge

After Phase A, no more OTA is needed for normal mod installation or dictionary changes.
```

Why the owner bridge matters

It gives you a safe one-shot root script mechanism from the normal USB-visible /user storage. The marker is deleted before the script runs, which avoids an automatic reboot loop if a script fails.

5. Obtain and verify the official V1.3 OTA image

Do not put the official firmware in a public GitHub repository. Each user should obtain the matching V1.3 image from OBOOK/support or from their own legitimate update/recovery source.

Verify the file

In WSL:

```
md5sum obook5-v1.3.img
```

The exact image used for this research produced:

```
8ced2385925ac03b0f6a82aa73a8e7bc  obook5-v1.3.img
```

Confirm that it is a gzip-compressed tar package

```
file obook5-v1.3.img
tar -tzf obook5-v1.3.img | head
```

This OTA is an overlay package, not a raw full-disk image. It contains application files, scripts, version metadata, and an internal checksum manifest. Not every library installed on the device has to be present in that OTA overlay, so the guide exports the live reader libraries from the device after the bootstrap bridge is installed.

Useful stock paths inside the package

```
image/opt/Qtopia/lib/libeinkfbreader.so.1.0.0
script/upgraded.sh
script_ext/upgraded.sh
upgrade_files.chksum
image/opt/Qtopia/home/version
```

6. Build the one-time owner bridge OTA

The public kit includes a builder script named tools/build_owner_bridge_ota.py. It takes your own official V1.3 image and creates a modified OTA locally.

Run the builder

```
python3 tools/build_owner_bridge_ota.py   obook5-v1.3.img   build/obook5-v1.4-owner-bridge.img   --
version V1.4
```

What the builder changes

- Appends a small owner bridge installer to script/upgraded.sh.
- Applies the same block to script_ext/upgraded.sh.
- Installs /home/obook-owner-boot-hook.sh.
- Backs up /etc/init.d/rcS before modifying it.
- Adds a background call to the owner hook in rcS.
- Updates the local synthetic version to V1.4.
- Recalculates upgrade_files.chksum.
- Repacks the OTA.
- Creates build/obook5-v1.4-owner-bridge.img.md5.

About V1.4

V1.4 here is a local lab label, not an official OBOOK release. The real official base remains V1.3.

Why two checksum layers matter

The updater checks the external MD5 of the complete image, and the unpacking flow also uses upgrade_files.chksum for individual files. Both must match the modified package.

7. Redirect the OBOOK updater to your local OTA server

The OBOOK updater requests an HTTP endpoint on port 8085:

```
http://service.obook.com.cn:8085/readerapi.aspx?version=...&model=epd86s2e
```

For a controlled owner installation, make that hostname resolve to your own PC on the local network.

7.1 Start a controlled local network

1. Turn on Windows Mobile Hotspot, or use another network where your PC can act as the local server.
2. Connect the OBOOK5 to that network.
3. Find the PC address visible from clients. In the tested Windows hotspot setup it was 192.168.137.1.

7.2 Override DNS for the OBOOK network

Configure the hotspot/gateway DNS so that:

```
service.obook.com.cn  ->  192.168.137.1
```

In the original controlled Windows setup, a local hosts/DNS override on the gateway path was used. If your hotspot does not forward that override to clients, use a local DNS proxy or router DNS rewrite. The requirement is simple: when the OBOOK requests service.obook.com.cn, it must reach your PC.

7.3 Start the local OTA server

```
python tools\ota_server.py ^
  --host-ip 192.168.137.1 ^
  --image build\obook5-v1.4-owner-bridge.img ^
  --version V1.4
```

The server listens on TCP 8085 and serves three things: readerapi.aspx, the image, and the image .md5 file.

7.4 Trigger the update

4. Leave the server running.
5. On the OBOOK, open the system update / Check upgrade function.
6. Watch the server console.
7. You should see a request for /readerapi.aspx with model=epd86s2e.
8. Accept the offered update.
9. Let the device complete the update and reboot.

If the server sees no request

Do not keep pressing update. Fix DNS/network routing first. The most useful proof is the request appearing in the Python server console.

8. Install and verify the owner startup bridge

The bootstrap OTA modifies the boot path once so future scripts can be delivered through ordinary USB/MTP storage.

What gets installed

```
/home/obook-owner-boot-hook.sh
/home/rcS.pre-obook-tools.backup
```
and rcS receives:
```
/home/obook-owner-boot-hook.sh &
```

How the one-shot mechanism works

```
if /user/ENABLE_OBOOK_STARTUP exists
AND /user/obook-startup.sh exists:

    delete ENABLE_OBOOK_STARTUP first
    run /user/obook-startup.sh as root
    write logs
    sync
```

Verify the bridge

10. Connect the OBOOK to the PC by USB.
11. At the root of the OBOOK storage, copy a harmless test obook-startup.sh and ENABLE_OBOOK_STARTUP.
12. Reboot.
13. After boot, ENABLE_OBOOK_STARTUP should be gone.
14. Check obook-boot-hook.log and obook-startup.log in the USB-visible storage.

Once this works, the OTA phase is finished. All later installations can use this one-shot bridge.

9. Export the stock reader libraries from the device

After the owner bridge is working, export the exact live libraries from your own device. This is the preferred public workflow because libdictionaryview is not necessarily present in the OTA overlay package.

```
1. Copy device/export_stock_libraries.sh to the OBOOK root as obook-startup.sh.
2. Copy ENABLE_OBOOK_STARTUP to the root.
3. Reboot.
4. The bridge copies both live stock libraries to:
   /user/OBOOK_STOCK_EXPORT/
5. Reconnect USB and copy that folder to your PC as stock/.
```

Verify the exported libraries

```
md5sum stock/libeinkfbreader.so.1.0.0
md5sum stock/libdictionaryview.so.1.0.0
```

Required values:

```
97f249f7a36b773582e78e80896e993b  libeinkfbreader.so.1.0.0
e18ae4624cb2bdf1edc041f203757b5d  libdictionaryview.so.1.0.0
```

Do not skip hash verification
The patch scripts use fixed offsets. The hashes are the compatibility check that makes those offsets meaningful.

10. Build the Search -> Dictionary patch

This patch changes the reader-side Search action so it creates/reuses the stock DictFetchScrWord helper and passes the selected QString to DictFetchScrWord::searchWord(). It also changes the visible action label from Search to Dictionary.

Build

```
mkdir -p build

python3 tools/apply_search_to_dictionary.py   stock/libeinkfbreader.so.1.0.0
build/libeinkfbreader.dictionary.so.1.0.0
```

Expected output

```
OK: build/libeinkfbreader.dictionary.so.1.0.0
MD5=86cd2535d141d13cb7abe4fd60746326
```

Verify manually

```
md5sum build/libeinkfbreader.dictionary.so.1.0.0
```

What happens under the hood

- The script refuses an unknown input hash.
- It assembles the small MIPS hook with clang targeting mipsel-linux-gnu.
- It replaces the tested search handler.
- It patches the live UI label source to Dictionary.
- It extends the ELF RW load / .bss by four bytes for a persistent helper pointer.
- It verifies the final tested MD5 before writing the output.

11. Install Search -> Dictionary

Prepare the three files

On the PC, copy/rename the installer:

```
device/install_search_to_dictionary.sh  ->  obook-startup.sh
```

Then copy these three files to the ROOT of OBOOK internal storage:

```
libeinkfbreader.dictionary.so.1.0.0
obook-startup.sh
ENABLE_OBOOK_STARTUP
```

Run the one-shot installation

15. Safely disconnect USB.
16. Reboot the OBOOK.
17. The installer replaces the active reader library and creates a backup.
18. The OBOOK should reboot itself one additional time.

Backup created

```
/user/OBOOK_BACKUP_libeinkfbreader.so.1.0.0
```

First test

For the first test, the installer points the stock dictionary configuration to the known-good built-in English -> Chinese dictionary:

```
/opt/image/opt/Qtopia/data/dictdb/default_ec0.dict
```

19. Open an English book.
20. Long-press a simple word such as house, book, or good.
21. The selection menu should now show Dictionary.
22. Tap Dictionary.

At this stage, the dictionary window should open. The next phase makes the selected word populate and search automatically.

12. Build the selected-word autofill patch

The stock dictionary view contains a broken handoff: DictFetchScrWord::searchWord() passes the selected QString to DictionaryView::fetchWord(), but that firmware implementation effectively ignores the QString and immediately calls search().

The validated fix

```
OLD
selected word
    -> fetchWord(selected)
    -> fetchWord ignores selected text
    -> search()

NEW
selected word
    -> setWord(selected)
    -> search()
    -> original stock showFullScreen path
```

Build

```
python3 tools/apply_autofill_v2.py   stock/libdictionaryview.so.1.0.0
build/libdictionaryview.autofill-v2.so.1.0.0
```

Expected MD5

```
02ef37b7d69995475b8cf5406ed4b053
```

This patch is intentionally small and preserves the stock window setup / geometry logic.

13. Install autofill v2

Prepare

```
device/install_autofill_v2.sh  ->  obook-startup.sh
```

Copy these three files to the ROOT of OBOOK storage:

```
libdictionaryview.autofill-v2.so.1.0.0
obook-startup.sh
ENABLE_OBOOK_STARTUP
```

Install

23. Safely disconnect USB.
24. Reboot.
25. The installer verifies the stock libdictionaryview hash.
26. It saves a stock backup.
27. It installs the validated autofill library.
28. The device reboots one additional time.

Backup created

```
/user/OBOOK_BACKUP_libdictionaryview.so.1.0.0
```

Test

29. Open a book.
30. Select a word.
31. Tap Dictionary.
32. The dictionary window should open.
33. The selected word should already be in the search field.
34. The lookup should already have run.

Validated end-to-end result
select word -> Dictionary -> autofill selected text -> automatic lookup -> result

14. Verify the complete dictionary flow

Before adding large custom dictionaries, prove that the code path works with the built-in known-good dictionary.

Suggested smoke test

- English word: house
- English word: book
- English word: good

Expected behavior

```
1. Long-press word
2. Dictionary appears in selection menu
3. Tap Dictionary
4. Dictionary window opens
5. Selected word is already inserted
6. Search runs automatically
7. A result is displayed
```

If this exact flow works

The firmware integration is finished. From this point onward, adding languages is a data-management task, not a firmware-hacking task.

15. Dictionary database format

An OBOOK-compatible customer .dict file is a SQLite 3 database with this table:

```
CREATE TABLE dict (
    word varchar (128) primary key,
    content varchar(255)
);
```

Conceptual example

```
word        content
----------- --------------------------------
house       casa; vivienda
book        libro
maison      casa; vivienda
bonjour     hola; buenos dias
```

The UI also accepts simple HTML-like content such as:

```
<b>house</b><br><i>noun</i>: casa, vivienda
```

Important

- word is the exact lookup key.
- word is unique because it is the primary key.
- If one word has several meanings, combine them inside content.
- Use UTF-8 in source datasets.

16. Create a new dictionary

The public kit includes dictionary/convert_to_obook_dict.py.

CSV example

```
word,content
house,casa; vivienda
book,libro
computer,computadora; ordenador
```

Convert

```
python dictionary/convert_to_obook_dict.py   MyDictionary.csv   MyDictionary.dict
```

Supported inputs

- CSV
- TSV
- JSON
- JSONL / NDJSON

Different column names

```
python dictionary/convert_to_obook_dict.py source.csv MyDictionary.dict   --word-field term   --
content-field definition
```

Add lowercase aliases

```
python dictionary/convert_to_obook_dict.py source.csv MyDictionary.dict   --lowercase-aliases
```

Inspect before installing

```
python dictionary/inspect_obook_dict.py MyDictionary.dict
python dictionary/inspect_obook_dict.py MyDictionary.dict --word house
```

17. Install a custom dictionary

Large dictionaries should live physically on /user, not on the small /data partition.

Storage design

```
REAL FILE
/user/OBOOK_DICTIONARIES/MyDictionary.dict

STOCK-VISIBLE LINK
/data/.词典/MyDictionary.dict
        ->
/user/OBOOK_DICTIONARIES/MyDictionary.dict
```

Prepare the installer

35. Open dictionary/install_and_select_dictionary_TEMPLATE.sh.
36. Change DICT_NAME="MyDictionary.dict" to the exact filename you are installing.
37. Save/copy the edited script as obook-startup.sh.

Copy to OBOOK root

```
MyDictionary.dict
obook-startup.sh
ENABLE_OBOOK_STARTUP
```

Reboot

The script moves the database to /user/OBOOK_DICTIONARIES, creates the symlink under /data/.词典, updates /home/Settings/dictionary.conf, and reboots.

Active configuration

```
[global]
default=/data/.词典/MyDictionary.dict

[dictionary]
selected=/data/.词典/MyDictionary.dict
```

18. Switch between installed dictionaries

If a dictionary is already in /user/OBOOK_DICTIONARIES, you do not need to copy the large database again.

Use the switch template

38. Open dictionary/switch_dictionary_TEMPLATE.sh.
39. Set DICT_NAME to the exact installed filename.
40. Save it as obook-startup.sh.
41. Copy obook-startup.sh and ENABLE_OBOOK_STARTUP to the OBOOK root.
42. Reboot.

Example

```
DICT_NAME="French-Spanish.dict"
```

The script updates the symlink/configuration and selects that database.

19. Merge multiple dictionaries

A combined database is convenient if you read several languages and do not want to switch dictionaries manually.

Example

```
python dictionary/merge_obook_dicts.py Combined.dict   EN=English-Spanish.dict   FR=French-
Spanish.dict   ES=Spanish-Spanish.dict
```

Result

If the same headword exists in more than one source, the merged content contains labeled sections, for example EN and FR.

Pros

- One active database.
- No firmware language detection needed.
- Shared words can show multiple interpretations.

Cons

- The file is larger.
- Some words may show more information than you want.

20. What to do with a pre-existing .dict file

The filename extension alone is not enough. Different dictionary systems use .dict for completely different internal formats.

Case 1 - it is already an OBOOK-compatible SQLite database

```
SQLite 3
  table: dict
  columns: word, content
```

If the schema matches, install it directly.

Case 2 - it is SQLite, but has another schema

Example:

```
CREATE TABLE entries (
    headword TEXT,
    definition TEXT
);
```

Read headword and definition, then write a new OBOOK database with word and content.

Case 3 - it is not SQLite

StarDict is a common example. A StarDict package may contain .ifo, .idx and .dict files. Its .dict file is not the same thing as an OBOOK SQLite .dict.

Quick checks

```
python dictionary/inspect_obook_dict.py SomeDictionary.dict
```

If Python reports 'file is not a database', it is not SQLite. If it opens but the expected dict table is missing, it needs conversion.

General conversion rule

```
original format
    -> extract headword + definition
    -> CSV / JSONL or direct Python mapping
    -> OBOOK SQLite:
       word + content
```

21. Morphology, plurals, conjugations, case and aliases

The mod hands the selected text to the dictionary exactly as selected. It does not automatically lemmatize every language.

Example

```
selected: maisons
lookup:   "maisons"
```

If the database contains only maison, the lookup may fail.

Useful aliases

```
maison     casa
maisons    casas; plural of maison

go         ir
goes       ir; third-person singular
went       ir; past tense
gone       ir; past participle
going      ir; gerund
```

Case

A dataset containing House may not behave like house in every exact-lookup situation. Lowercase aliases are useful.

Accents

Keep the source in UTF-8 so French and Spanish characters remain correct.

```
é è ê ë à â ç ñ á í ó ú ü
```

Data quality matters
For this reader, a dictionary with inflected-form aliases can feel much more complete than a lemma-only database with a similar number of definitions.

22. Recovery and rollback

Recovery was designed into the installation flow. The owner bridge runs before the reader application, so a broken reader library can still be replaced if the main system continues to boot and expose USB storage.

Restore Search -> Dictionary library

43. Copy device/restore_reader.sh to the OBOOK root.
44. Rename/copy it as obook-startup.sh.
45. Copy ENABLE_OBOOK_STARTUP again.
46. Reboot.

Restore stock dictionary view

47. Copy device/restore_autofill_v2.sh to the OBOOK root.
48. Rename/copy it as obook-startup.sh.
49. Copy ENABLE_OBOOK_STARTUP.
50. Reboot.

Backups

```
/user/OBOOK_BACKUP_libeinkfbreader.so.1.0.0
/user/OBOOK_BACKUP_libdictionaryview.so.1.0.0
/home/rcS.pre-obook-tools.backup
```

Why the one-shot marker is safe

ENABLE_OBOOK_STARTUP is deleted before obook-startup.sh runs. If the script crashes or exits badly, it does not automatically rerun on every boot.

Do not improvise if the device no longer boots

Low-level BootROM recovery is outside this public procedure. Stop and use an appropriate manufacturer recovery path rather than blindly flashing partitions.

23. Recommended GitHub repository structure

```
OBOOK5-offline-dictionary/
|
|-- README.md
|-- LICENSE
|-- docs/
|   |-- TESTED_HASHES.md
|   `-- public-reproduction-guide.pdf
|
|-- tools/
|   |-- build_owner_bridge_ota.py
|   |-- ota_server.py
|   |-- apply_search_to_dictionary.py
|   |-- search_to_dictionary_hook.S
|   `-- apply_autofill_v2.py
|
|-- device/
|   |-- export_stock_libraries.sh
|   |-- install_search_to_dictionary.sh
|   |-- restore_reader.sh
|   |-- install_autofill_v2.sh
|   |-- restore_autofill_v2.sh
|   `-- ENABLE_OBOOK_STARTUP
|
`-- dictionary/
    |-- convert_to_obook_dict.py
    |-- inspect_obook_dict.py
    |-- merge_obook_dicts.py
    |-- install_and_select_dictionary_TEMPLATE.sh
    |-- switch_dictionary_TEMPLATE.sh
    `-- example_dictionary.csv
```

The repository should NOT contain

- Official OBOOK .img firmware.
- Extracted OBOOK .so libraries.
- Pre-patched proprietary OBOOK .so libraries unless you have explicit redistribution rights.
- Commercial dictionary data you do not have permission to redistribute.

Why build scripts are better

Users prove they have the exact compatible firmware locally, and the patcher refuses a mismatched hash. This is both cleaner technically and safer for redistribution.

24. What you should and should not publish

Good public material

- Your original Python scripts.
- Your original shell scripts.
- The MIPS hook source you wrote.
- Technical documentation and hashes.
- Instructions for exporting exact stock libraries from a user's own device and extracting files from their own official firmware where applicable.
- Dictionary conversion tools.
- Open-data source instructions with proper licensing notes.

Keep separate / do not bundle by default

- OBOOK proprietary firmware.
- OBOOK proprietary binaries.
- Private device logs that contain unnecessary identifiers.
- Third-party lexical datasets without checking their license.

Responsible framing

Describe the OTA redirection as a controlled owner-network bootstrap for the tested device. Do not frame the procedure as a way to modify somebody else's reader or intercept another person's update traffic.

If you discuss the lack of cryptographic authentication observed in the OTA path, keep that security disclosure separate from the dictionary feature and distinguish the controlled proof from claims about universal remote exploitability.

25. Quick reference checklist

One-time bootstrap

- [ ] Exact OBOOK5 / epd86s2e.
- [ ] Official V1.3 image MD5 matches.
- [ ] Build owner bridge OTA.
- [ ] Start local DNS redirect + OTA server.
- [ ] Trigger Check upgrade.
- [ ] Confirm owner bridge runs one-shot scripts.

Reader patch

- [ ] Stock libeinkfbreader hash matches.
- [ ] Build final patched library.
- [ ] Final MD5 = 86cd2535d141d13cb7abe4fd60746326.
- [ ] Copy payload + obook-startup.sh + marker.
- [ ] Reboot and confirm Dictionary menu.

Autofill

- [ ] Stock libdictionaryview hash matches.
- [ ] Build autofill-v2.
- [ ] Final MD5 = 02ef37b7d69995475b8cf5406ed4b053.
- [ ] Copy payload + installer + marker.
- [ ] Reboot and confirm selected word is searched automatically.

Custom dictionary

- [ ] SQLite dict(word, content).
- [ ] Test with inspect_obook_dict.py.
- [ ] Install under /user/OBOOK_DICTIONARIES.
- [ ] Symlink under /data/.词典.
- [ ] dictionary.conf points to the selected database.

Appendix A. Tested hashes and important paths

```
Official V1.3 OTA image
MD5 8ced2385925ac03b0f6a82aa73a8e7bc

Stock reader library
/opt/image/opt/Qtopia/lib/libeinkfbreader.so.1.0.0
MD5 97f249f7a36b773582e78e80896e993b

Patched Search -> Dictionary reader
MD5 86cd2535d141d13cb7abe4fd60746326

Stock dictionary view
/opt/image/opt/Qtopia/lib/libdictionaryview.so.1.0.0
MD5 e18ae4624cb2bdf1edc041f203757b5d

Patched autofill-v2 dictionary view
MD5 02ef37b7d69995475b8cf5406ed4b053

Active dictionary configuration
/home/Settings/dictionary.conf

Customer dictionary path recognized by firmware
/data/.词典/

Large user storage used by the mod
/user/OBOOK_DICTIONARIES/
```

Appendix B. What the binary patches change

Search -> Dictionary

The working reader patch replaces the tested QtApplicationWindow::search(const QString&) path with a MIPS handler that creates one persistent DictFetchScrWord helper and calls:

```
DictFetchScrWord::searchWord(selectedQString)
```

The visible Search label used by the selection UI is changed to Dictionary. The patch also extends the ELF writable memory area by four zero-initialized bytes for the persistent helper pointer.

Autofill v2

In the tested stock libdictionaryview, the selected QString reached fetchWord(), but fetchWord() did not use it as a word setter. The validated minimal patch changes the existing searchWord() path so it effectively performs:

```
getView()
setWord(selectedQString)
getView()
search()
original cleanup / showFullScreen()
```

This keeps the original dictionary window lifecycle instead of replacing the whole function.

Appendix C. Owner bridge logic

```
boot
 |
 v
BusyBox init
 |
 v
/etc/init.d/rcS
 |
 +--> /home/obook-owner-boot-hook.sh &
          |
          | wait for /user mount
          |
          | if both exist:
          |   /user/ENABLE_OBOOK_STARTUP
          |   /user/obook-startup.sh
          |
          | delete marker FIRST
          |
          `--> run obook-startup.sh as root
```

This bridge is the reason the public workflow needs only one OTA. Every later change can be staged through normal USB/MTP storage.

Appendix D. Dictionary schema and sample data

```
CREATE TABLE dict (
    word varchar (128) primary key,
    content varchar(255)
);
```

Plain content

```
house -> casa; vivienda
book  -> libro
```

HTML-like content

```
<b>maison</b><br>
<i>French -> Spanish</i><br>
1. casa<br>
2. vivienda
```

Minimal Python example

```
import sqlite3

con = sqlite3.connect("MyDictionary.dict")
con.execute(
    "CREATE TABLE dict ("
    "word varchar (128) primary key, "
    "content varchar(255)"
    ")"
)
con.execute(
    "INSERT INTO dict(word, content) VALUES (?, ?)",
    ("house", "casa; vivienda")
)
con.commit()
con.close()
```

End of guide